

Neural Network for handwritten Digit Recognition: Multiclass 0-9

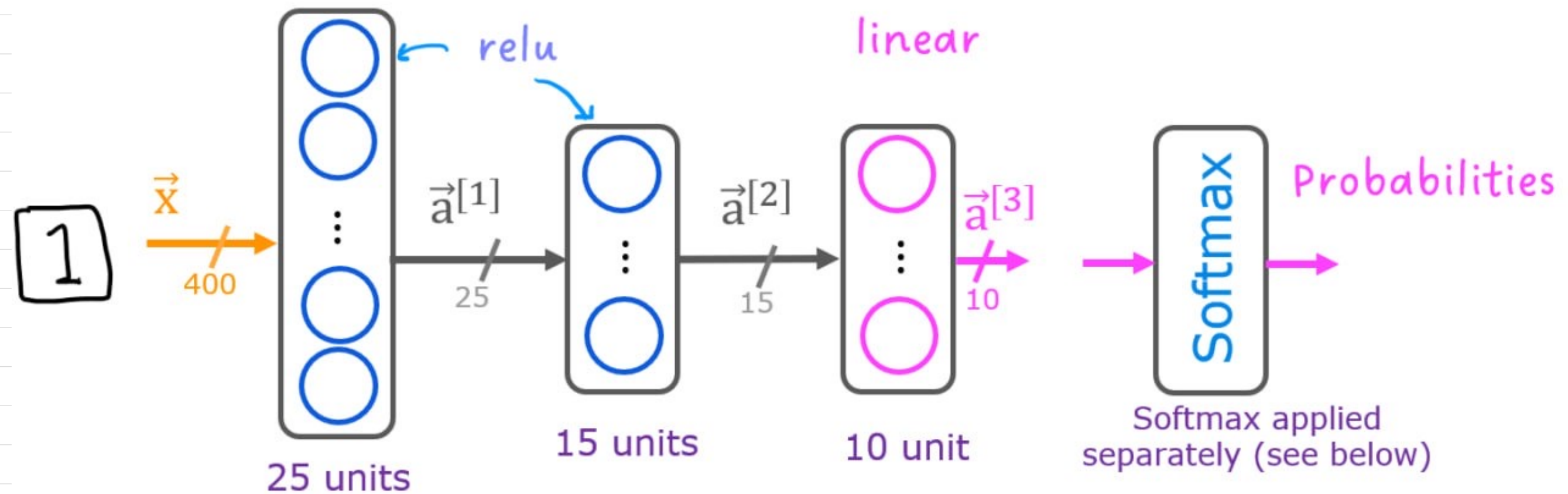
Dataset:

- Contains 5000 examples of handwritten Digit 0-9
- Each example is 28-Pixel x 28-Pixel flattened into 784-dim Vector
- Each example becomes single Row in X
- This gives 5000 x 784 matrix X

$$X = \begin{bmatrix} \text{Pixel 1} , \text{Pixel 2} , \text{Pixel 784} \end{bmatrix}$$

5000
Example
row

Model



- input size $20 \times 20 \rightarrow (400,)$
- Params of our model

$$W_1 = (400, 25) \quad , \quad b_1 = (25,)$$

$$W_2 = (25, 15) \quad , \quad b_2 = (15,)$$

$$W_3 = (15, 10) \quad , \quad b_3 = (10,)$$

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 25)	10,025
dense_1 (Dense)	(None, 15)	390
dense_2 (Dense)	(None, 10)	160

Total params: 10,575 (41.31 KB)

Trainable params: 10,575 (41.31 KB)

Non-trainable params: 0 (0.00 B)

```
[25]: W1,b1,W2,b2,W3,b3 = model.get_weights()
```

```
[27]: print(f"W1 shape = {W1.shape}, b1 shape = {b1.shape}")
      print(f"W2 shape = {W2.shape}, b2 shape = {b2.shape}")
      print(f"W3 shape = {W3.shape}, b3 shape = {b3.shape}")
```

```
W1 shape = (400, 25), b1 shape = (25,)
W2 shape = (25, 15), b2 shape = (15,)
W3 shape = (15, 10), b3 shape = (10,)
```

The following code:

The following code:

- defines a loss function, `SparseCategoricalCrossentropy` and indicates the softmax should be included with the loss calculation by adding `from_logits=True`
- defines an optimizer. A popular choice is Adaptive Moment `Adam`

```
[31]: model.compile(  
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),  
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),  
)  
  
model.fit(  
    X,y,  
    epochs = 40  
)
```

```
Epoch 1/40  
157/157 ————— 1s 798us/step - loss: 1.6917  
Epoch 2/40  
157/157 ————— 0s 739us/step - loss: 0.6726  
Epoch 3/40  
157/157 ————— 0s 741us/step - loss: 0.4639  
Epoch 4/40  
157/157 ————— 0s 743us/step - loss: 0.3781  
Epoch 5/40  
157/157 ————— 0s 740us/step - loss: 0.3298  
Epoch 6/40  
157/157 ————— 0s 763us/step - loss: 0.2994  
Epoch 7/40  
157/157 ————— 0s 732us/step - loss: 0.2734  
Epoch 8/40  
157/157 ————— 0s 738us/step - loss: 0.2579  
Epoch 9/40  
157/157 ————— 0s 738us/step - loss: 0.2579
```


Prediction

To make a prediction, use Keras `predict`. Below, `X[1015]` contains an image of a two.

```
In [26]: image_of_two = X[1015]
display_digit(image_of_two)

prediction = model.predict(image_of_two.reshape(1, 400)) # prediction

print(f" predicting a Two: \n{prediction}")
print(f" Largest Prediction index: {np.argmax(prediction)}")
```



```
predicting a Two:
[[ -7.99  -2.23   0.77  -2.41 -11.66 -11.15  -9.53  -3.36  -4.42  -7.17]]
Largest Prediction index: 2
```